

ARM CREATE 上海站 · 云 AI 专场

Agent 时代的执行底座

CubeSandbox 的技术甜点与工程挑战



Cube Sandbox

Just For Agent: A New Infrastructure Layer

Apache 2.0

E2B SDK 兼容

冷启动 < 60ms

单实例 < 5MB

OpenCloudOS 9

ARM64 原生

Like Xu · Tencent Cloud

2026 年 9 月 9 日 13:00–13:30 · 上海西岸美高梅酒店

github.com/TencentCloud/CubeSandbox · cubesandbox.com



arm



Agenda

时代在召唤：Agent 正把「执行」变成基础设施的第一性问题

01 Agent 时代的负载特点与典型场景

四个旧假设失效 · 两类 Agent · 三类负载

02 Cube 如何满足 Agent 底座的核心要求

来时路 · 八条设计原则 · 六个技术甜点

03 开源成绩单与业界影响力

公开数据 · 生态位 · 一战成名

04 核心技术解析：v0.1 → v0.7

版本演进 · 全栈锁优化 · 排查案例

05 在企业生产中落地 CubeSandbox

能力清单 · 部署路径 · 适用边界

06 共建 Agent Infra：OpenCloudOS 9 + ARM

CPU 信号 · OpenCloudOS 9 + ARM · 开源协作

一句话主线：CubeSandbox 是一款基于 RustVMM 与 KVM 构建的高性能、高密度、高并发、开箱即用的开源安全沙箱服务。

01

Agent 时代的负载特点与典型场景

当 AI 从「对话」变成「自主执行」，被重构的不是模型，是整条执行链路

四个设计假设同时失效

个人 Agent 向左，服务 Agent 向右

三类基本负载：工具执行 / Harness / 服务底座

四个沿用多年的设计假设，在 Agent 场景同时失效

不是「跑得久一点的微服务」，而是一种新工作负载（自主性、不确定性、工具与环境依赖、非确定性有状态演进）

失效的假设	为什么失效	新的 Infra 要求
默认信任 外部隔离，内部放行	行为不可预测代码，「内部放行」等于不拦 - Amazon Kiro 删除了自家生产环境（2025.12） - OpenClaw 安全危机（2026 年初）	零信任：默认禁止，每次行为都要授权，全链路审计
无状态 用完即走	状态丢失即能力归零，环境 / Skills / 配置 / 记忆才是核心价值 - 上下文窗口有限，长任务中逐渐遗忘初始约束	状态是一等公民：可快照、可回滚、可持久化
短生命周期 以请求为单位	路径不可预测，可能连续运行数天到数周 - Loop/Graph Engineering 开发范式	长生命周期：以 Agent 实例为单位管理，暂停而非销毁
操作者 = 持有人 谁登录谁负责	Agent 代表用户行动，但不是用户本人、凭证散落在多实例 - Moltbook 150 万 Agent 凭证泄露（2026.01）	委托授权：区分「谁发起」与「谁执行」，凭证不由执行方持有

规则驱动 → 自主决策

系统层不确定 → 逻辑层不确定

环境为执行服务 → 环境是 Agent 的一部分

四条一起失效，就必须重做底座。

重新定义生产力：个人 Agent 向左，服务 Agent 向右

两类 Agent 并存互补，但对执行环境的要求几乎相反 —— 底座要同时满足两个方向

个人 Agent · 站在用户侧

- ▶ **角色** 数字助手：写邮件、查资料、写报告、写代码、操作 Web / App / PDF
- ▶ **意图来源** 理解用户个人意图，为个人创造或完成任务
- ▶ **形态** 一人一实例，环境长期存在，Skills / 配置 / 记忆持续积累
- ▶ **对底座的要求** 硬隔离 + 高密度（每人一个环境）· 状态不丢（快照 / 回滚）· 闲时不烧钱（暂停恢复）

服务 Agent · 站在服务侧

- ▶ **角色** 持续在线的服务：电商分析、智能客服、售前知识库、人力 / 法务咨询
- ▶ **意图来源** 遵循服务提供者与管理者的意图，对数据输出承担责任
- ▶ **形态** 连接 CRM / 知识库 / 工单 / OA / ERP，服务大量用户或被 Agent 调用
- ▶ **对底座的要求** 高可用 7×24 · 数据权限与内容审核 · 防滥用 · 全链路可审计

两者交汇处，正是底座最吃力的地方

- ▶ **完整的工作环境是前提** 「让 AI 能跑起来干活的，一套完整的工作环境也是必须的 —— 文件系统、工具调用、长记忆、反馈循环」
- ▶ **交汇场景** 个人 Agent 拆解任务、整理资料、生成初稿；需要客户数据、报价规则、合同模板时，发请求给服务 Agent 承接，由后者核实身份、对数据权限把关、防御滥用
- ▶ **落到底座上** 这一步跨越了信任边界 —— 对应委托授权、出站治理与全量审计

两类 Agent 的划分与交汇场景出自汤道生《和 AI 一起长跑的这两年》（腾讯云公众号，2026）第四节。

Agent-centric AI：三类基本负载

第一类要「快」，第二类要「记得住」，第三类要「服务本身可分叉」

① 工具执行环境

- 多类型环境：代码执行、浏览器、桌面、移动端操作
- 要求高并发拉起能力，接近本地调用
- 要求高资源利用率：一台服务器承载上千并发实例
- Agentic RL 训练与评测：海量原生镜像（SWE-bench、Android / OS World）与并发 rollout

② Agent Harness 长时任务

- 本质是有状态服务，持续产生中间态与持久化产物
- 快速的状态保存 / 恢复，匹配暂停恢复与克隆回滚
- 任务可能连续运行数天到数周，中间不能丢状态
- 典型：Manus / OpenClaw、Work / CodeBuddy、Webhook 触发

③ 面向 Agent 的服务底座（多租户）

- 服务直接成为 Agent 训练与推理循环的一部分
- 对快速启停、分支探索与回滚提出要求
- 传统服务如何零代码改造成 Agent 友好
- 使用者从人变成 Agent，突发性与实验性成为常态

案例：DeepSeek Harness（2026-08 开源，「一切皆插件」）

- ▶ **沙箱本身就是插件** 模型、工具、会话、沙箱 { code, browser, computer, mobile }、存储、循环、调度、UI 全部由插件提供，可替换重组
- ▶ **自带沙箱的边界** 基于 Linux bwrap / Landlock、macOS Seatbelt、Windows 受限令牌，分 read-only / workspace-write / danger-full-access 三档
- ▶ **官方明确的限制** 该层主要约束文件副作用；网络与进程可见性不由它解决，partial enforcement 不等于完整隔离
- ▶ **对 Cube 的含义** Harness 负责编排，隔离边界可以换成 VM 级 —— 这正是沙箱的插入点

负载三分类来源：InfoQ 专访《腾讯云为何重构 Agent 沙箱？》(2026-08-12)。DeepSeek Harness 来源：官方仓库 README 与源码文档（Developer Preview）。

02

Cube 如何满足 Agent 底座的核心要求

不是在微服务架构上打补丁，而是把全栈定向优化了

四类痛点与 Cube 的答案

来时路：从 Serverless 到 Agent Runtime

八条设计原则 · 四层架构 · Cube MicroVM

击溃友商的六个技术甜点

四类痛点，指向同一个结论：Agent 缺一个可信的运行底座

降低 “Agent错误执行” 的成本, 进而提升效率

痛点	具体表现	Cube 的答案
缺少可控的安全执行边界	Prompt 注入 · 高权限 Agent · 敏感与版权数据 · 传统容器共享宿主机内核	KVM 级隔离：独立 Guest Kernel + 文件系统 / 网络 / CPU / 内存隔离，配合审计与回滚
缺少轻量、快速启动、高并发的沙箱	RL 训练需要海量镜像 + 并发启动 + 决策 rollout；容器 / VM 冷启动在秒级到分钟级，而 LLM / RL 与 Agent 交互需要百毫秒	模板快照启动 + 资源池化：端到端 < 60ms，单机数千实例
缺少上下文敏感、极致弹性的资源交付	Agent 与上下文强亲和（记忆 / 状态）；生命周期长短不一，带来成本浪费与资源碎片	状态一等公民：快照 / 克隆 / 回滚 + AutoPause 闲时释放、请求唤醒
缺乏统一协议与抽象层	生态蓬勃但未出现标准，阻碍 Agent Infra 可移植与规模化；自建沙箱成本高	E2B SDK 兼容 + Volume 插件框架 + Apache 2.0，接进来和走掉都是改配置

痛点归纳自 CubeSandbox 团队公开分享与官方文档；右列机制均可在仓库中找到对应组件。

Cube 不是 Agent 火了之后才开始做的

底层是一套 2023 年前后为 Serverless 建设、已在生产磨合两三年的基础设施



从 Serverless 到 Agent: Cube 系统的八条设计原则

每一条都能在仓库里找到对应代码，欢迎探索

设计原则	做法	由此得到
① 分布式调度 + 单机装箱	控制面只做分发，单个计算节点独立完成创建与装箱	加节点即加并发，控制面不成为瓶颈
② 资源池化 + 单机闭环	磁盘（reflink）、网卡（TAP）、计算配额提前池化，创建不依赖外部控制面	百毫秒交付，不受 IaaS 控制面链路影响
③ 前后端解耦	入网络、后端资源、出网络三处无状态化，支持热替换	资源可替换，快照可跨实例复用
④ 快照恢复 + lazy load	模板快照含容器状态，恢复即 ready；未访问区域延迟加载	冷启动只发生一次，发生在模板制作阶段
⑤ 资源复用 + 按需申请	内核 / rootfs / 镜像全局只读共享，写才 CoW；磁盘 reflink 超卖	单实例开销 < 5MB，单机数千实例
⑥ 全栈锁优化	从应用到内核逐层消除全局锁与同步点	数百并发下不出现秒级时延抖动
⑦ 原生安全	独立 Guest Kernel + RustVMM 极简设备模型，seccomp / cgroup 兜底	边界在硬件层，不在 namespace
⑧ 复用虚拟机资源	直接复用成熟的存储与网络体系：virtio-pmem / fs / blk、eBPF vswitch	不重造存储与网络，少走一遍弯路

原则框架来源：《从 Serverless 到 Agent: Cube 系统的一些设计思考》（腾讯云开发者社区，2026-05）；实现细节对应 README 架构章节与 v0.1–v0.6 Changelog。

四层架构：从 E2B 兼容的 API，一路管到 KVM

控制面无状态、数据面节点本地 —— 这是「加节点即加并发」的结构前提



来源：docs/architecture/overview.md（Control Plane / Data Plane 划分）与 README Architecture。控制面无状态、Redis 为唯一事实来源；数据面按节点自治。

甜品桌 - Cube MicroVM：把「弹性」和「密度」同时做到极致

四个方向上的关键技术点 —— 这是六个甜点背后共同的工程底子

高弹性 · 快速启动

- MVM Kernel + RootFS 极致裁剪
- MVM 快照优化
- Reflink Copy
- 启动过程不依赖 IaaS 控制面流程

高弹性 · 低时延

- 启动流程锁优化
- 启动过程性能隔离
- 核心资源提前池化准备
- 创建 / 销毁只更新母机本地状态

高密度 · 内存共享

- MVM Template Fork MVM, Kernel 与 RootFS 内存 COW
- 所有共享的只读文件全物理机共享, 全局只占一份内存
- MVM 内部只读存储 DAX bypass pagecache

高密度 · 存储共享

- 业务只读镜像 (代码) 采用 virtio-fs, 全局存储一份
- 可写临时存储 Reflink copy + COW
- CPU / 内存 / 存储规格可灵活分配, 避免装箱碎片

MVM 网络：高密度与高弹性如何兼得

- ▶ **配置固化统一** 所有子机内部网络配置 (IP / 路由 / ARP) 保持一致并提前静态配置, 生产子机时无需动态配置与 ARP 学习
- ▶ **快照友好** 网络配置一致, 也方便统一快照内容, 加速基于快照的启动
- ▶ **预创建 / 复用** TAP 设备池化复用, 减少频繁创建销毁的系统开销
- ▶ **host-local IP** 基于母机做 overlay, TAP + eBPF 实现 vswitch 与外部互通; 创建销毁只需重置 TAP 并更新母机本地内存路由, 对 VPC 控制面无依赖

甜点 ① 低时延：模板快照启动，冷启动只发生一次

把启动链路搬进模板制作阶段，请求到来时只是恢复，不是启动

容器冷启动路径



Cube 模板快照启动路径（同一时间轴）



容器快照解决了什么

- ▶ **快照包含容器状态** 容器状态恢复 · 管控通路重建 · 容器信息映射 · 容器磁盘恢复
- ▶ **恢复即服务 ready** 服务百毫秒内交付，不再走一次服务启动
- ▶ **并发性能大幅提升** 时延降低 50%+ · 开销降低 50%+

为什么整条链路可以省掉

- ▶ **资源提前池化** 磁盘、网卡、计算配额单机闭环预备
- ▶ **网络配置固化** 创建销毁只改母机本地路由
- ▶ **只读共享 + 延迟加载** 内核、rootfs、镜像全局共享，未访问区域不申请

实测口径（裸金属）

- ▶ **单并发** 60ms 端到端交付可服务沙箱
- ▶ **50 并发** 平均 67ms · P95 90ms · P99 137ms
- ▶ **生产环境** 单台物理机数分钟内拉起数万沙箱

数据来源：README Benchmarks、v0.1.0 Changelog、docs/blog/posts/2026-06-01；一键压测：tests/perf/cubebench.sh。

甜点 ② 高密度：只读内容全机共享一份

资源复用 + 按需申请，让第 1000 个沙箱几乎不增加成本

只读共享：全物理机一份

- ▶ **共享对象** 虚拟机内核、rootfs、容器镜像、模板快照
- ▶ **实现方式** 只读盘走 virtio-pmem + DAX，绕过 guest pagecache
- ▶ **业务镜像** virtio-fs 全局存储一份

写时复制：按需申请，延迟申请

- ▶ **内存** 快照以 mmap private 只读共享，写才 CoW；恢复后未访问区域不申请
- ▶ **磁盘** XFS reflink 池化 + CoW 按需分配，支持超卖
- ▶ **脏页追踪** soft-dirty 增量快照，首次全量之后只抓脏页

极致轻量：把开销一层层削掉

- ▶ **RustVMM** 语言内存安全 + 简化设备模型
- ▶ **virtiofsd 改造** 线程化去 vhost；文件 direct、目录 cache；线程裁剪与聚合
- ▶ **进程合并** shim 与 hypervisor 合并，IPC 变函数调用
- ▶ **OS 优化** 裁剪 rootfs、优化虚拟机内核；日志异步 + buffer

< 5MB

单实例内存开销

沙箱规格 ≤32GB 实测

数千

单机沙箱实例数

内核共享 + CoW，小规格

128 → 8MB

按需访问的实际写入

规格 128MB → 快照 31MB → 实际写 8MB

-35%

网络 P50 时延 (v0.4)

35.3 → 23.1ms；P99 -41%

内存开销出自 README Benchmarks；网络数字出自 v0.4.0 Changelog (BMI5 / Xeon Platinum 8255C / kernel 6.6.119)；按需访问口径出自公开设计分享。

甜点 ③ 快照 / 回滚 / 克隆：给 Agent 一台时光机和一门分身术

CubeCoW 把「存档 / 读档」做成基础设施能力，也就是 Agent 的编程原语

快照 Snapshot

- 把 running 沙箱的内存、设备状态、磁盘整体 dump 成独立文件
- 生命周期与源沙箱解耦，源沙箱销毁后仍可用
- 快照 ID 可直接当模板，批量启动新沙箱

克隆 Clone

- 从一个 running 沙箱派生 N 个完全独立副本
- 继承性：内存、文件、连接一致；隔离性：副本间物理隔离；连续性：源沙箱继续运行
- 内置并发控制与失败自动清理，不留孤儿沙箱

回滚 Rollback

- 原地恢复到任意快照，内存与文件系统完全还原
- sandbox_id 与沙箱对象不变，不需要重连重建
- 配合 CubeProxy 上游连接刷新，回滚后数据面不卡在旧连接

实现要点

- ▶ **O(1) 快照与克隆** XFS reflink 的 FICLONE ioctl —— 快照只改元数据，不做字节拷贝
- ▶ **扁平快照模型** 删除任一快照不影响其他快照
- ▶ **增量脏页追踪** 只持久化自上次快照以来变更的匿名页，未变页继续 reflink 共享 (#389)
- ▶ **恢复正确性** VSOCK 恢复时重置 (#388)；virtio 设备 resume 时 kick 队列并注入中断 (#1084)

它解决哪些问题

- ▶ **RL Harness** 同一状态并行探索多条轨迹，失败轨迹一键回退
- ▶ **Coding Agent** 事件级存档：改坏了回到上一个 checkpoint，而不是重建环境
- ▶ **批量实验** 从「依赖已装好、已跑到第 3 步」的状态克隆 N 份
- ▶ **成本** AutoPause 空闲暂停，下一个请求到来时自动唤醒

来源：docs/architecture/overview.md (CubeCoW 章节)、v0.3.0 / v0.5.0 Changelog、docs/guide/snapshot-rollback-clone.md。

快照 / 克隆 / 回滚有什么用：以强化学习训练为例

同一个环境状态上探索多条轨迹，是 RL Harness 的基本动作



没有快照能力时

- ▶ **串行重跑** 每条轨迹都要从零装依赖、跑到第 N 步，时间成倍
- ▶ **并行也不省** 开 N 个沙箱，每个都要重复一次完整的环境准备
- ▶ **失败代价高** 轨迹走坏了只能销毁重建，中间状态全部丢失
- ▶ **镜像压力** 海量镜像反复拉取，分发成为瓶颈

有了快照能力

- ▶ **环境准备只做一次** 基线状态做成快照，之后所有轨迹从快照起跑
- ▶ **克隆近似免费** reflink 元数据操作 + 内存 CoW，第 N 份副本几乎不额外占资源
- ▶ **失败可回退** 回滚到上一个 checkpoint，sandbox_id 不变，不需要重连
- ▶ **成功可分叉** 从任意一个好状态继续派生，形成搜索树

配套能力

- ▶ **镜像加速** 块级去重 + 按需加载，海量镜像不再是瓶颈
- ▶ **官方镜像内置** SWE-bench 系列、Android World、OS World 等评测环境
- ▶ **并发控制** 克隆内置 concurrency 限制与失败自动清理
- ▶ **跨节点 (v0.7)** 快照可在其他物理节点恢复，调度不再被单机绑定

来源：docs/guide/snapshot-rollback-clone.md、v0.3.0 Changelog、docs/guide/tutorials/examples.md (RL 相关示例)。

甜点 ④ 强隔离：边界在硬件层，Key 不进沙箱

原生安全 + 零信任出网：内核态先决定能不能出去，L7 再决定带什么出去



来源：v0.4.0 / v0.5.0 / v0.6.0 Changelog、docs/guide/security-proxy.md、docs/guide/network-policy.md、docs/architecture/network.md。

甜点 ⑤ 可迁移：迁移成本 = 改两个环境变量

原生兼容 E2B SDK；隔离级别拉满的同时，启动速度没有变差

```
import os
from e2b_code_interpreter import Sandbox
# 只改这两行：把 E2B Cloud 指向自己的 CubeSandbox 集群
os.environ["E2B_API_URL"] = "http://<控制节点IP>:3000"
os.environ["E2B_API_KEY"] = "e2b_000000"

with Sandbox.create(template=os.environ["CUBE_TEMPLATE_ID"]) as sbx:
    print(sbx.run_code("import platform; print(platform.machine())"))
# x86_64 或 aarch64 — 同一份业务代码，两种 CPU 架构
```

兼容面与缺口

▶ 接入

Python / Go / Node SDK、CLI、REST

▶ 已对齐

文件系统 API、PTY、Volume、lifecycle

▶ 质量门禁

仓库自带 sdk_compat E2E 套件，兼容性回归进 CI

▶ 缺口

官方 Roadmap 把「完全 drop-in」列为进行中目标

维度	Docker 容器	传统虚拟机	CubeSandbox
隔离级别	低（共享内核 Namespace）	高（独立内核）	极高（独立内核 + eBPF 网络隔离）
启动速度	约 200ms	秒级，端到端 5–10s	毫秒级（< 60ms）
内存开销	低（共享内核）	高（完整 OS）	低（极限裁剪，< 5MB）
部署密度	高	低	极高（单机数千实例）

对比表中 Docker 与传统虚拟机的数字为量级参考，非同机对照实验；Cube 数字出自官方裸金属基准。仓库自带一键压测工具，欢迎复现。

甜点 ⑥ 自动暂停恢复：闲时释放资源，用时毫秒唤醒

沙箱空闲后自动暂停并销毁 VM；请求到来时快速恢复为可用实例



来源：v0.5.0 Changelog (AutoPause / AutoResume)、docs/guide/lifecycle.md、docs/architecture/overview.md (cube-lifecycle-manager)。

03

开源成绩单与业界影响力

四个月的公开数据，以及数据背后真正值得看的东西

开源四个月：Star / Fork / PR / Commit

一战成名：生产迁移效果

开源四个月：携手 Arm 等多家厂商，共建社区，依旧澎湃

比总量更值得看的是节奏 —— 每两到三周一个功能版本



生态位

- ▶ **CNCF Landscape** 收录于 AI-Native Infra / Workload Runtime 分类
- ▶ **Arm 官方** 以 Sponsor 身份出现在仓库，并深度共研 ARM64 原生支持
- ▶ **OpenClaw** 2026-07，创始人主动把 Cube 接入 provider 体系，与 E2B、Modal 并列
- ▶ **上游回馈** containerd 引用锁优化已合入 containerd 2.0.2；PVM 内核改动已开源进 OpenCloudOS Kernel
- ▶ **Cube 100** 面向前 100 个把 Agent 跑上生产的团队征集可公开、可度量的案例

怎么读这些数字

- ▶ **Star 只说明关注度** 不说明生产可用度，下一页和第 5 章才回答那个问题
- ▶ **Fork 与 PR 更有信息量** 说明有人真的在改它
- ▶ **贡献者口径要小心** GitHub 计数含 bot 与匿名提交，不同口径能差十几人
- ▶ **贡献者在涨** v0.3 22 人 → v0.4 15 人 → v0.5 26 人 → v0.6 31 人；v0.6 之后至今 55 人参与



数据来源：GitHub REST API，采集于 2026-08-26；版本 commit 与贡献者数出自各版本 Changelog；OpenClaw 一项出自 InfoQ 专访 (2026-08-12)。⚠ 此类数据持续变化。

一战成名

某大模型应用的代码沙箱，从原有架构迁移到 Cube 后的三类收益



04

核心技术解析

v0.1 → v0.7

每一版对应 Agent 落地路上一道具体障碍

版本地图：v0.1 → v0.7

v0.7：快照跨越节点，控制面高可用

全栈锁优化：内核 11 项 · 应用与系统 16 项

五个月 12 个 release，每一版解决一个真问题

版本演进：从「跑得快」到「跑得稳」「跑得广」「管得住」

版本	日期	解决的问题	关键特性
v0.1.0	2026.04.20	有没有一个敢跑未知代码、又不慢的沙箱	60ms 冷启动 · <5MB 开销 · 独立 Guest Kernel · E2B 兼容 · CubeVS eBPF 网络隔离
v0.2.0	2026.05.07	没有裸金属的人怎么试	PVM 部署（普通云服务器即可） · Web 管理控制台 · 内核改动开源进 OpenCloudOS Kemel
v0.3.0	2026.06.02	Agent 的状态怎么存、怎么回退、怎么分叉	CubeCoW 快照引擎 · soft-dirty 增量内存快照 · Go SDK · AgentHub（Preview）
v0.4.0	2026.06.14	凭证怎么不进沙箱，升级后怎么知道要重建哪些模板	CubeEgress L7 安全代理 · 凭证注入与审计 · 节点组件版本矩阵 · 网络 P50 -35%
v0.5.0	2026.07.03	闲置沙箱的成本，以及只能跑 x86 的问题	AutoPause / AutoResume · ARM64 全栈原生 · Terraform 一键集群 · 网络加固
v0.6.0	2026.07.24	已有 K8s 平台的团队怎么接，存储怎么插自己的	Kubernetes 部署（Preview） · E2B 兼容 Volume 框架 · 模板别名 · 节点隔离 · CubeOps
v0.7.0	2026.08	快照绑在单节点上，控制面还不能随便挂	跨节点暂停 / 快照恢复 · CubeS3lvol 远程 CoW · 组件多版本共存 · CubeOps 接管节点管理

来源：docs/changelog/index.md 与各版本 Changelog。v0.7.0 内容为已合入 master 的特性，正式清单与日期以发布时的 Changelog 为准。

v0.7：快照跨越节点，控制面高可用

把沙箱的抽象从单机提升到集群 —— 从「单机很强」走向「集群很强」

方向	做了什么	编号
跨节点暂停 / 快照恢复	以 S3 / COS 为后端把暂停包与快照外置，沙箱可在其他物理节点恢复；配套跨节点恢复兼容性检查。节点故障时沙箱可在别处快速拉起	#1489 / #1409 / #1331
CubeS3Ivol 远程 CoW	以对象存储为后端、通过 NVMe-oF 暴露的块存储，本地盘只放 WAL 与元数据日志；支持快照导出、跨节点传输与克隆	#1490 / #1507
组件多版本共存	同一节点上保留多个组件版本，老快照在组件升级后仍能恢复；cube-agent 与 guest image 解耦	#1316 / #1234
控制面职责下沉	节点管理迁到 CubeOps，支持安全节点删除；CubeMaster localcache 数据竞争修复	#1384 / #1202 / #1376
出网策略热更新	原地更新沙箱出网策略并重新评估在途连接；支持任意自定义 L7 端口与 scheme	#1399 / #1347
存储与 SDK 补齐	内置 MinIO S3 Volume 后端；Go / Node SDK 补齐 Volume API；模板可后设别名	#1408 / #1269 / #1277 / #1120

更远一点三个方向

- Kubernetes 原生化：Helm → CRD / Operator
- 可观测下沉 OS 层：系统调用 / 文件 / 进程
- 服务 Agent Teams：共享上下文与产物

编号为已合入 master 的 PR / Issue（git log v0.6.0..HEAD，采集于 2026-08-26：203 commits / 55 位参与者）。⚠ 截至采集时尚未打 tag，正式特性清单与发布日期以 v0.7.0 Changelog 为准。

60ms 的平均值不难，难的是数百并发下不抖

全栈锁优化：从应用一路挖到内核，逐层消除全局锁与同步点

网络与并发

- TAP fd 三级获取 (fast / hot / recovery) : P50 -35%、P99 -41%、吞吐 +8%
- EnsureNetwork / ReleaseNetwork 的 TOCTOU 竞争：改用 per-sandbox 守卫通道，64 goroutine 压测 -race 干净
- BPF SNAT 误用伪首部，导致跨节点端口映射回包 checksum 损坏
- netlink 相关全局锁：TAP 池化并集中初始化

快照与虚拟化

- 快照恢复后 virtio 设备 livelock：resume 时 kick 队列并注入中断 (#1084)
- pause 时排空并 flush virtio-blk 在途 IO (#1193)
- soft-dirty 检测改为真实 pagemap 往返探测 (#1115)
- v0.5 快照运行时锁重构，消除高并发死锁

系统与上游

- containerd 引用锁在高并发下秒级等待：改为失败立退、上层全局重试 —— 已合入 containerd 2.0.2
- Cubelet 独立 mount namespace，并从 private 改 slave 让宿主 mount 事件单向传播
- PVM 嵌套虚拟化内核改动已开源进 OpenCloudOS Kernel
- shim 用 Rust 重写并与 hypervisor 合并进程，IPC 调用变函数调用

这些问题的共性

- ▶ **都是规模才暴露的** mount 传播、containerd 引用锁、netlink、cgroup 加入、内存规整 —— 单机几十个实例时它们全都不存在
- ▶ **都不出现在 feature list 上** 但它们决定 P99，而 P99 才是生产上被投诉的那个数
- ▶ **这是开源的价值** 自建团队要重新踩一遍；而这些已经被生产流量踩过，并写进了 Changelog

来源：v0.2–v0.5 Changelog 与公开 PR #1084 / #1115 / #1193；containerd 侧见 containerd 2.0.2 发布记录；框架见《Cube 系统的一些设计思考》1.6 节。

全栈锁优化 · 内核侧：11 个真实的同步点

每一条都是「单机几十个实例时根本不存在」的问题

epmutex 争抢

主机嵌套 epoll 并发增删触发全局锁争抢，优化应用设计避免嵌套实现

rtnl_lock 争抢

主机 tap 操作涉及 rtnl_lock 全局锁，多进程 tap 初始化销毁触发严重竞争；tap 池化并集中初始化

task_list 竞争

并发虚拟机销毁触发线程销毁过程中 task_list 锁争抢；对线程销毁流程做流控，异步销毁虚拟机

synchronize_cpu 等待

mvm 进程加入 cgroup 需等 rcu 同步导致时延抖动；异步设置实例 cgroup，避免阻塞创建流程

kvm 内核线程启动抖动

kvm nxlpage 启动需要同步加入 cgroup，阻塞虚拟机创建交付；修改配置可关闭 nxlpage 线程

kvm 内存 compact

kvm 使用连续物理内存，高密度场景内存同步 compact 触发动抖；优化避免使用物理连续内存

memcg reparent_file

高并发场景将主机共享 pagecache reparent，避免影响虚拟机 cgroup 内存使用

srcu race

深挖高并发场景主机内核 srcu race 故障，推动解决 srcu 内部异常卡住问题

mount_lock 争抢

避免主机侧 overlayfs 争抢，消除主机侧 mount / umount，容器路径直通虚拟机内部挂载

ebpf map-in-map

主机内核 ebpf 使用 map-in-map 触发 rcu 同步，引入时延抖动；修改程序避免使用 map-in-map

fdtable 争抢

多线程 fdtable 扩展触发进程 fdtable 锁争抢，优化程序逻辑提前进行 fdtable 扩展

框架见《从 Serverless 到 Agent：Cube 系统的一些设计思考》1.6「全栈锁优化」；部分条目对应的修复可在公开 Changelog 与 PR 中找到痕迹。

全栈锁优化 · 应用与系统侧：16 项收口

从 hypervisor 进程内部，到宿主机的 systemd 与内存水位

Hypervisor 日志异步

高并发日志写 IO 抖动；日志模块重构，异步 + 缓冲彻底解决 IO 竞争

Hypervisor GSI 更新

设备恢复时 batch 操作 KVM 做 GSI 更新，大幅减少调用次数

Hypervisor setgroups

多线程下 setgroups 需等线程同步；提前 setgroup 到程序入口

Shim / Hypervisor 进程合并

Rust 重写 shim 并与 hypervisor 合并，IPC 调用转为函数调用，降低内存开销与启动时延

Hypervisor 设备限速

设备引入限速能力，针对多实例场景兜底，减少虚拟机相互影响

virtiofsd dentry 主动回收

实现 dentry cache 管理与主动回收，同步修改虚拟机内核实现 dentry 缩容

Seccomp 预编译

规则预编译成 bpf 字节码，避免 VM 启动时重复构建

Golang forklock

引入底层 fork lock 优化，避免 fork 并发调用时锁争抢

Systemd 挂载点跟踪消除

systemd 监控挂载点在高并发下开销巨大；通过独立 namespace 部署 cube 组件消除抖动

虚拟机缓存策略

修改 virtiofs cache 策略：目录项 cache、文件内容不 cache，减少虚拟机内存占用

虚拟机 pmem 优化

只读 rootfs / 基础镜像采用 pmem 设备 dax 挂载，无虚拟化后端开销

洋葱进程冲突消除

减少外部巡检进程对 shim 的探测，消除虚拟机创建时延抖动

Memory water boost

wmark 水位抬高会概率触发直接内存回收导致抖动；关闭 boost 配置

KVM pml

非迁移场景关闭 pml，大幅优化大规格内存 MVM 创建时延

虚拟机串口优化

virtio-console 替代串口，大幅降低串口输出触发的大量 vmexit

磁盘内容预读

对明确访问的文件区域预读并锁定，用少量 pagecache 消除 IO 抖动

同上：框架见《Cube 系统的一些设计思考》1.6 节。上半部分为 hypervisor / 运行时侧，下半部分为宿主机系统侧。

05

在企业生产中落地 CubeSandbox

企业落地卡的从来不是「能不能跑」，是「跑起来之后怎么管、怎么升、怎么过审」

开箱即用、部署即可审计

多条部署路径与 Day-2 运维

开箱即用、部署即可审计

左边是拿来就能用的，右边是 Agent 沙箱在网络上无法绕开的硬需求

开箱即有的产品能力

- ▶ **生命周期** 沙箱创建 / 销毁 / 暂停 / 恢复，快照管理
- ▶ **存储** 文件系统 host-mount 与 Volume 插件框架 (Binary / RPC 两种形态)
- ▶ **权限** API-KEY 管理与权限控制
- ▶ **协议兼容** 兼容 E2B 沙箱协议，存量代码零改动接入
- ▶ **多语言 SDK** Python / Go / Node
- ▶ **镜像** 支持自定义镜像与模板别名
- ▶ **控制台** WebUI 可视化：总览 / 节点 / 沙箱 / 模板 / 版本

AI Agent 沙箱的网络层要解决的核心问题

- ▶ **隔离性** 沙箱之间、沙箱与宿主机之间的网络必须完全隔离
- ▶ **可控性** 精确控制每个沙箱可以访问哪些外部服务 (IP / 域名 / 协议)
- ▶ **可观测性** 所有出站流量必须可审计、可追溯
- ▶ **透明性** 对 Agent 代码零侵入，不需要修改应用逻辑
- ▶ **高性能** 网络策略不能成为千级并发的瓶颈
- ▶ **Cube 的答案** CubeVS 在内核态 (eBPF) 做隔离与过滤，CubeEgress 在 L7 做策略、凭证注入与审计；两者协同，沙箱流量无法绕过检查通道

来源：README 产品能力章节、docs/guide/{volume-plugin, authentication, network-policy, webui}.md、docs/architecture/network.md。

多条部署路径，为生产落地护航

选路径的依据是手上有什么硬件、已经有什么平台

路径	适合谁	前置条件	状态
PVM 云服务器	想先试、没有裸金属	普通云服务器 + OC-PVM 内核（目前仅 x86_64）	推荐入门
裸金属 / 物理机	追求极限性能与密度	支持 KVM 的 x86_64 或 aarch64 物理机	生产可用
Terraform 一键集群	在腾讯云上要生产集群	一条 create.sh 供给 VPC / MySQL / Redis / CFS / TCR / TKE / 计算节点	v0.5 起
Kubernetes (Helm)	已有 K8s 平台，要原生升级灰度	TKE / 标准 Kubernetes / k3s	Preview

升级与回退

三方 .env 配置合并、fail-fast 预检、变更前自动备份；可关闭启动时自动建表

版本一致性

节点组件版本矩阵 + 模板副本兼容检查，升级后哪些模板要重建一目了然

节点治理

节点隔离（cordon 语义）、安全下线、指定节点创建、调度打分可配

可观测

Prometheus 指标、容器 stdout/stderr 落盘可读、E2B 兼容 metrics API

高可用中间件

外接 MySQL / Redis（含 Sentinel）、PostgreSQL 元数据、CFS 下多副本

成本控制

AutoPause 释放配额比例可配、软删除定时清理、磁盘 reflink 超卖

装完第一件事：打开 `http://<控制节点IP>:12088`，确认节点 Ready → 模板商店装一个官方预设 → 新建沙箱看实时日志。来源：README Quick Start、v0.4–v0.6 Changelog、docs/guide/。

06

共建 Agent Infra OpenCloudOS 9 + ARM

算力供给正在双轨化，而 Agent 负载恰好是高核数、常态高并发、能效敏感的那一类

行业信号：沙箱写入 CPU 产品定义

与 ARM 开源协作：Bring CubeSandbox to Arm64

Cube + OpenCloudOS 9 + ARM：不只是兼容目标

Agent 沙箱不再是应用层选择，已经写入 CPU 产品定义

两家处理器厂商同时把「并发沙箱环境数」当作核心指标

Arm AGI CPU：为编排层设计



- **定位** Arm 首款量产芯片，基于 Neoverse CSS V3；面向持续推理、决策、执行的 agentic 负载
- **职责** 编排算力、管理加速器、同时协调数千个 Agent
- **规格** 136 × Neoverse V3 / 每核 2MB L2 / 3.5GHz / 300W；12 × DDR5-8800、96 lane PCIe Gen6、CXL 3.0
- **机架密度** 10U 双节点参考服务器 272 核；液冷机架 336 颗 CPU、45,696 核
- **厂商口径** 整机架性能 > 2× 最新 x86 系统（Arm 估算）

NVIDIA Vera：The CPU for agents



- **定位** 明确面向模型之外的 CPU 工作：代码执行、工具调用、沙箱、分析、数据管道、编排
- **规格** 88 个自研 Olympus 核 / 176 线程；LPDDR5X 带宽至 1.2TB/s、容量至 1.5TB
- **厂商口径** agentic 沙箱负载较领先 x86 快至 1.8×；沙箱环境性能提升至多 80%
- **机架** Vera CPU Rack 集成至多 256 颗 CPU，运行 22,500+ 并发环境
- **论点** 「并发环境数」成了机架级指标，与 tokens/s 并列

这对我们意味着什么

- **硬件在往这个方向走** 沙箱的冷启动时延与单机密度，从工程指标变成了数据中心的效率指标
- **软件底座必须跟上** 拿到高核数与高带宽之后，能不能真的塞进数千个隔离环境，取决于运行时

arm.com/products/cloud-datacenter/arm-agi-cpu · nvidia.com/en-us/data-center/vera-cpu（采集于 2026-08-26）。两家数字均为厂商自述或估算口径，非第三方实测。

与 ARM 开源协作：Bring CubeSandbox to Arm64

从「编译通过」到「原生可用」，中间隔着多层兼容性重写

门槛	具体问题	怎么解的
构建工具链	Go / Rust / OpenSSL / libseccomp 的下载路径与环境变量硬编码 x86_64	Dockerfile.builder 参数化 TARGETARCH；多架构 manifest；一条 docker buildx 出双架构镜像
eBPF 代码生成	BPF 的 //go:generate 硬编码 -target amd64，缺 arm64 vmlinux.h，且需要 clang ≥ 14	改为 -target \$GOARCH；补 arm64 BTF 头；builder 装 clang-14；.o 不再入库，构建时生成
Guest Kernel	x86_64 的 kernel config 不能直接复用	独立 kernel-oc9.aarch64.config + 专门的 vmlinux 构建目标，支持在 x86 上交叉编译
Hypervisor 与引导	Guest→Host 的就绪 / 关机通知走 x86 PIO 端口（0x680），Arm 上不存在；x86 靠 SeaBIOS 引导；seccomp 系统调用号不同	SysCtrl 从 PIO 改 MMIO（0x0903_0000）；改用 UEFI 固件；seccomp 白名单按架构对齐（mkdir→mknod，补 faccessat2）

移植之外的两个成果

- **调优成果** Guest Kernel 裁剪后镜像体积约 -25%、内存分配约 -40%、沙箱创建 P50 约 -8%；多 NUMA 机器建议把控制面组件绑到同一 NUMA 节点
- **协作成果** 双方交叉 review 在合入前抓出 3 个真实回归；commits 由 Arm 与腾讯双方工程师 review、测试并签署 —— 这种审视强度，才是移植原生可用的基石

arm

来源：v0.5.0 Changelog（ARM64 Native Support）、官方博客《Cube Sandbox 正式支持 Arm 架构》(2026-07-08)。优化数字为约数，公开材料未给出完整测试环境口径。

Cube + OpenCloudOS 9 + ARM：不只是兼容目标，是下一环

算力供给正在双轨化，而 Agent 负载恰好是高核数、常态高并发、能效敏感的那一类

Arm 侧：已在主线，长期支持

arm

- ▶ **双轨化是真实预算** Graviton / Axion / Cobalt / Arm AGI CPU 与国产 Arm 持续迭代落地
- ▶ **负载天然匹配** Agent 从突发编排变成常态高负载，核数与能效成为定义性指标
- ▶ **已在主线** ARM64 全栈原生支持在 v0.5.0 合入，覆盖编译、构建、启动、部署全链路
- ▶ **Arm 深度参与** 以 Sponsor 身份出现在仓库，并直接贡献原生支持

OpenCloudOS 9 侧：共享上游



- ▶ **社区定位** 由腾讯及合作伙伴共同发起的中立、开放操作系统社区，继承腾讯在操作系统与内核领域十余年积累，平等支持各硬件平台
- ▶ **AI Native OS** AI 全栈生态、安全补丁修复、安全可信底座
- ▶ **官方推荐 OS** Cube 部署文档首选推荐 OpenCloudOS 9（内核 6.6）
- ▶ **配置已在仓库** kernel-oc9.x86_64.config / kernel-oc9.aarch64.config
- ▶ **生产级 PVM** PVM 嵌套虚拟化内核改动已开源进 OpenCloudOS Kernel
https://mirrors.opencloudos.tech/opencloudos/9.6/extras/x86_64/os/Packages/

为什么值得你参与

- ▶ **走的人多了，路就宽了** OpenCloudOS 9 + aarch64 是官方推荐配置，但公开的实践与数据仍然最薄
- ▶ **门槛比想象的低** Guest 内核支持交叉编译（在 x86 上就能产出 Arm 内核镜像）；多架构镜像一条 docker buildx 出；性能调优可借助 Arm Performix 工具链
- ▶ **硬件正在到位** Arm AGI CPU 与 NVIDIA Vera 都把并发沙箱环境写进了产品定义，aarch64 上的沙箱运行时会越来越被需要

总结 CubeSandbox: 为 Agent 而生

01

高度适配: 自主执行的 Agent 正在重新定义基础设施

默认信任、无状态、短生命周期、操作者=持有人 —— 四个假设同时失效, 对应零信任、状态一等公民、长生命周期、委托授权四个新要求。

02

技术甜点: 极速启动 · 持续并发 · 高密部署 · 时光机 · 分身术 · 强隔离 · 可迁移

模板快照启动、只读共享与按需申请、分支探索与回滚、基于 KVM 的独立内核、密钥不进沙箱、E2B / Cube SDK 双向兼容。

03

生产就绪, 开源共建

高并发不抖、多种部署形态、快照跨节点、控制面持续加固、网络全链路可审计, 并与 Arm、OpenCloudOS 等伙伴共同推进。

★ 让每一个 Agent 都拥有定义良好、功能完善的安全运行空间 ★

Q & A · github.com/TencentCloud/CubeSandbox · cubesandbox.com



腾讯云

arm



OpenCloudOS



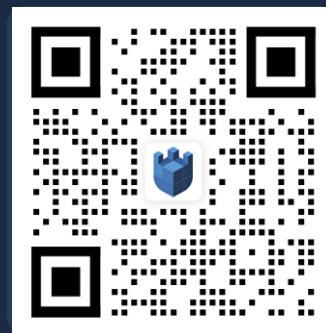
Cube Sandbox

Thank You

Cube — Empowering your AI agents

github.com/TencentCloud/CubeSandbox

cubesandbox.com · [contribution](#) · [community](#)



用户交流群



微信公众号



腾讯云

arm



OpenCloudOS